

Principals implementation plan

Issue: <https://github.com/electric-sql/electric/issues/4306>

Goal

Add **principals** as a first-class entity type so every action in the agents system traces to an owning identity.

Principals are entity streams addressed as:

```
/principal/user:kyle  
/principal/agent:ci-bot  
/principal/service:github  
/principal/system:framework  
/principal/system:dev-local
```

Inbound requests carry the principal in a trusted header set by edge/auth middleware:

Electric-Principal: user:kyle

In local/dev mode, missing headers default to:

system:dev-local

Because agents are pre-release, there is no backwards compatibility path for unauthenticated/no-principal requests. API routes should error if the request has no principal.

Request context

Move request identity from user-centric naming to principal-centric naming:

```
// before  
authenticatedUser?: AuthenticatedRequestUser
```

```
// after  
principal: Principal
```

All routes, including internal routes, should include a principal. Use one principal-aware entry point into the verbs instead of parallel unauthenticated/internal code paths.

There are no first-class “users” in the agents runtime; there are only principals, one kind of which may be user.

Creator field

Use **created_by** for the immutable entity creator/owner field. It stores a principal entity URL, e.g. /principal/user:kyle.

Principal model

Add a new module:

```
packages/agents-server/src/principal.ts
```

Types:

```
export type PrincipalKind = 'user' | 'agent' | 'service' | 'system'

export interface Principal {
  kind: PrincipalKind
  id: string
  key: string // `${kind}:${id}`
  url: string // `/principal/${kind}:${id}`
}
```

Header constant:

```
export const ELECTRIC_PRINCIPAL_HEADER = 'electric-principal'
```

Helpers:

```
export function parsePrincipalKey(input: string): Principal
export function principalUrl(key: string): string
export function principalKeyFromUrl(url: string): string | null
export function getPrincipalFromRequest(request: Request): Principal | null
export function getDevPrincipal(): Principal
```

Validation rules:

- Principal key is exactly {kind}:{id}.
- Exactly one colon is allowed.
- Kind is one of:
 - user
 - agent
 - service
 - system
- ID must be non-empty.
- ID must not contain /.
- ID should use a conservative character set, e.g.:

[A-Za-z0-9._@-]+

Examples:

user:kyle	✓	
agent:ci-bot	✓	
service:github	✓	
system:framework	✓	
system:dev-local	✓	
user:clerk:user_123	✗	additional colon
user:/kyle	✗	slash
admin:kyle	✗	unknown kind

Request principal extraction

Wire request extraction wherever the server builds TenantContext.

Likely files to inspect/change:

```
packages/agents-server/src/host.ts
packages/agents-server/src/routing/global-router.ts
packages/agents-server/src/entrypoint-lib.ts
packages/agents-server/src/dev-asserted-auth.ts
packages/agents-server/src/authenticated-user-format.ts
packages/agents-server/src/electric-agents-types.ts
```

The authenticated-user-format module may become obsolete or should be renamed/reworked as a principal formatter/parser.

Desired behavior:

```
const headerValue = request.headers.get('electric-principal')

const principal = headerValue
  ? parsePrincipalKey(headerValue)
  : isDevOrInsecure
    ? getDevPrincipal() // system:dev-local
    : null
```

If no principal exists, return an auth/invalid-request error.

As part of this, replace user-centric request auth names with principal-centric names:

- AuthenticatedRequestUser → AuthenticatedRequestPrincipal or just RequestPrincipal
- AuthenticateRequest should return a Principal/principal assertion, not a user object
- ctx.authenticatedUser → ctx.principal
- fields such as userId should become principal fields, e.g. principal.key, principal.url, principal.kind, principal.id

The agents server trusts this header. Auth middleware/proxy is responsible for setting it correctly.

Built-in principal entity type

Principals must be normal entities, so ensure a built-in entity type named principal exists.

Add to PostgresRegistry:

```
async ensureEntityType(et: ElectricAgentsEntityType):
  Promise<ElectricAgentsEntityType>
```

Behavior:

- Insert if missing.
- If present, return existing unchanged.
- Do not bump revisions on every server startup.

Seed at server/runtime startup:

```
await registry.ensureEntityType({
  name: 'principal',
  description: 'built-in principal entity',
```

```

inbox_schemas: {
  update_identity: principalUpdateIdentityMessageSchema,
},
state_schemas: {
  identity: principalIdentityStateSchema,
},
revision: 1,
created_at: now,
updated_at: now,
})

```

The principal entity type has one built-in state collection:

- `identity` — trusted profile/identity information for the principal.

The principal entity type has one built-in inbox message:

- `update_identity` — request to create/update the identity state row.

The principal entity type is immutable from user/API code. It is created and modified only by system code.

Principal identity state

Add built-in schema definitions for principal identity.

Identity state row:

```

const principalIdentityStateSchema = {
  type: 'object',
  additionalProperties: false,
  required: ['kind', 'id', 'key', 'url', 'updated_at'],
  properties: {
    kind: { enum: ['user', 'agent', 'service', 'system'] },
    id: { type: 'string' },
    key: { type: 'string' },
    url: { type: 'string' },
    display_name: { type: 'string' },
    email: { type: 'string' },
    avatar_url: { type: 'string' },
    auth_provider: { type: 'string' },
    auth_subject: { type: 'string' },
    claims: {
      type: 'object',
      additionalProperties: true,
    },
    created_at: { type: 'string' },
    updated_at: { type: 'string' },
  },
}

```

Identity state uses a single stable key:

identity/self

Update message schema:

```
const principalUpdateIdentityMessageSchema = {
  type: 'object',
  additionalProperties: false,
  required: ['identity'],
  properties: {
    identity: principalIdentityStateSchema,
  },
}
```

The `update_identity` inbox message is how principal identity is created/updated. Anyone can target a principal entity with this message shape at the protocol/schema level, but authorization must restrict who is allowed to send it.

In Electric Cloud, a built-in system entity should send `update_identity` when:

- a user logs in via Google/SSO/etc.
- a CI bot principal is created
- a service integration principal is provisioned
- identity/profile data changes in the upstream auth system

Non-system principals should not be allowed to send `update_identity` unless explicitly authorized by deployment-specific policy.

Persistence changes

Add migration:

`packages/agents-server/drizzle/0006_principals.sql`

SQL:

```
ALTER TABLE entities
  ADD COLUMN created_by text;

CREATE INDEX idx_entities_created_by
  ON entities (tenant_id, created_by);
```

Update Drizzle schema in:

`packages/agents-server/src/db/schema.ts`

Add to entities:

```
createdBy: text(`created_by`),
```

Update server types in:

`packages/agents-server/src/electric-agents-types.ts`

Add to `ElectricAgentsEntity`:

`created_by?: string`

Add to `PublicElectricAgentsEntity`:

`created_by?: string`

Add to `TypedSpawnRequest`:

`created_by?: string`

Update `toPublicEntity()` to include `created_by`.

Update registry in:

`packages/agents-server/src/entity-registry.ts`

- `createEntity()` writes `createdBy: entity.created_by ?? null`
- `rowToEntity()` reads `created_by`
- `listEntities()` accepts `created_by?: string`
- `listEntities()` filters on `entities.createdBy`

Update route list filtering in:

`packages/agents-server/src/routing/entities-router.ts`

Support:

GET `/_electric/entities?created_by=/principal/user:kyle`

Lazy principal materialization

Add to `EntityManager`:

async `ensurePrincipal(principal: Principal): Promise<ElectricAgentsEntity>`

Behavior:

1. Check `registry.getEntity(principal.url)`.
2. If found, return it.
3. If missing, create a principal entity at that URL.

Principal spawn details:

```
await this.spawn('principal', {
  instance_id: principal.key,
  args: {
    kind: principal.kind,
    id: principal.id,
    key: principal.key,
  },
  tags: {
    principal_kind: principal.kind,
    principal_id: principal.id,
  },
  created_by: principal.url,
})
```

On creation, also initialize identity/self with the built-in identity state:

```
{
  kind: principal.kind,
  id: principal.id,
  key: principal.key,
  url: principal.url,
  created_at: now,
  updated_at: now,
}
```

If trusted auth/profile claims are available during materialization, include the mapped fields in identity/self.

Need to avoid recursive principal creation. Either:

- Add an internal spawn option such as { skipPrincipalEnsure: true }, or
- Implement ensurePrincipal() using a lower-level helper that creates the entity without trying to ensure created_by first.

Recommended internal rule for created_by:

```
const createdBy = req.created_by ?? parentEntity?.created_by
```

This means child/worker agents inherit the initiating principal from their parent unless explicitly overridden.

For principal entities themselves, created_by can be their own URL:

```
/principal/user:kyle created_by=/principal/user:kyle
/principal/system:dev-local created_by=/principal/system:dev-local
```

Route behavior

File:

```
packages/agents-server/src/routing/entities-router.ts
```

Principal route materialization

Current withExistingEntity() returns 404 if the entity is missing. Adjust for principal URLs:

```
if (!entity && request.params.type === 'principal') {
  const principal = parsePrincipalKey(request.params.instanceId)
  const materialized = await ctx.entityManager.ensurePrincipal(principal)
  request.entityRoute = { entityUrl, entity: materialized }
  return undefined
}
```

This enables:

```
POST /_electric/entities/principal/user:bob/send
```

to create Bob's principal stream on first reference.

Spawn

In `spawnEntity()`:

1. Require `ctx.principal`.
2. Ensure the inbound principal exists.
3. Pass `created_by: ctx.principal.url` to `entityManager.spawn()`.
4. Use principal as the initial message sender.

Pseudo-code:

```
const principal = requirePrincipal(ctx)
await ctx.entityManager.ensurePrincipal(principal)

const entity = await ctx.entityManager.spawn(request.params.type, {
  instance_id: request.params.instanceId,
  args: parsed.args,
  tags: parsed.tags,
  parent: parsed.parent,
  dispatch_policy: dispatchPolicy,
  initialMessage: undefined,
  wake: parsed.wake,
  created_by: principal.url,
})

if (parsed.initialMessage !== undefined) {
  await ctx.entityManager.send(entity.url, {
    from: principal.url,
    payload: parsed.initialMessage,
  })
}
```

Send

In `sendEntity()`:

1. Require `ctx.principal`.
2. Ensure the inbound principal exists.
3. Default from to `ctx.principal.url`.
4. Reject client-supplied from if present and not equal to `ctx.principal.url`.

Recommended v1 security posture:

- HTTP send should not allow arbitrary from spoofing.
- Use `ctx.principal.url` as sender.
- Internal APIs/tools should also pass through the same principal-aware verb entry point.

So update route behavior to:

```
await ctx.entityManager.send(entityUrl, {
  from: principal.url,
  payload: parsed.payload,
  key: parsed.key,
```



```
    type: parsed.type,  
  })
```

Do not allow callers to assert arbitrary principals via body. The request/context principal is the sender.

Principal identity updates

Principal entities accept an `update_identity` inbox message, but ordinary principals must not be able to send it by default.

Route/send authorization should enforce:

- `update_identity` to `/principal/*` is allowed from built-in system principals.
- `update_identity` from non-system principals is rejected unless deployment policy explicitly allows it.
- Other messages to `/principal/*` can continue through normal send authorization.

This preserves the uniform send mechanism while letting Electric Cloud run a built-in system entity that creates/updates principals from trusted auth events.

Schedule/future-send

Future-send route currently accepts `from` in body.

For the same anti-spoofing reason, schedule routes should ignore/reject body `from` and use `ctx.principal.url`:

```
from: principal.url
```

Inter-principal messaging

Once principal entities are lazy-materialized, the existing send mechanism works:

```
POST /_electric/entities/principal/user:bob/send  
Electric-Principal: user:kyle  
Content-Type: application/json
```

```
{  
  "payload": { "text": "hello" }  
}
```

Result:

- `/principal/user:kyle` is ensured.
- `/principal/user:bob` is ensured.
- Bob's principal inbox receives a message with:

```
from: '/principal/user:kyle'
```

Handler/runtime context

Issue requirement:

Authorization v1: Handler decides which tools/functions to expose based on principal context.

Handlers need access to principal information.

Likely files to inspect/change:

```
packages/agents-runtime/src/create-handler.ts
packages/agents-runtime/src/setup-context.ts
packages/agents-runtime/src/types.ts
packages/agents-server/src/entity-manager.ts // enrichPayload()
```

EntityManager.enrichPayload() currently injects entity info into webhook payloads. Add:

```
entity: {
  ...,
  createdBy: entity.created_by,
},
principal: entity.created_by
? {
  url: entity.created_by,
  key: principalKeyFromUrl(entity.created_by),
}
: undefined,
```

Then expose this through runtime handler context as:

```
ctx.principal
ctx.entity.created_by
```

This enables handler-level authorization:

```
const tools = ctx.principal?.kind === 'user'
  ? userTools
  : serviceTools

await runAgent({ tools })
```

Authorization v1

Keep authorization flexible and handler-level.

What this implementation should do now:

- Identify the inbound principal.
- Persist owner/creator on spawned agents.
- Prevent routes from spoofing from in request bodies.
- Materialize principal streams lazily.
- Expose principal to handlers.

What this implementation should **not** do yet:

- Capability expressions in entity streams.

- Named capability sets.
- Delegation semantics.
- General cross-principal policy engine beyond the built-in `update_identity` restriction.
- Public sharing links or signed URLs.
- Principal garbage collection.

Tests

Principal parser tests

File:

`packages/agents-server/test/principal.test.ts`

Cases:

- `user:kyle` → `/principal/user:kyle`
- `agent:ci-bot` → `/principal/agent:ci-bot`
- `service:github` → `/principal/service:github`
- `system:framework` → `/principal/system:framework`
- `system:dev-local` → `/principal/system:dev-local`
- reject missing colon
- reject additional colon
- reject slash
- reject empty id
- reject unknown kind

Spawn records owner principal

- Send PUT `/_electric/entities/<type>/<id>` with `Electric-Principal: user:kyle`.
- Assert spawned entity has:

`created_by: '/principal/user:kyle'`

- Assert `/principal/user:kyle` exists.

Child spawn inherits created_by

Using `EntityManager.spawn()` directly:

1. Create parent with `created_by: '/principal/user:kyle'`.
2. Spawn child with parent and no explicit `created_by`.
3. Assert child has same `created_by`.

Send uses principal as from

- Create an entity.
- POST `/send` with `Electric-Principal: user:kyle` and no body from.
- Read stream.
- Assert inbox event value has:

`from: '/principal/user:kyle'`

Public send does not allow spoofed from

- POST /send with Electric-Principal: user:kyle and body from: '/principal/user:alice'.
- Assert the route rejects the request with 400/422.

Sending to unmaterialized principal creates it

- POST /_electric/entities/principal/user:bob/send
- Header: Electric-Principal: user:kyle
- Assert:
 - /principal/user:bob exists.
 - /principal/user:kyle exists.
 - Bob's inbox has from: /principal/user:kyle.

Principal identity is initialized

- Materialize /principal/user:kyle.
- Assert its state contains identity/self with:

```
{  
  kind: 'user',  
  id: 'kyle',  
  key: 'user:kyle',  
  url: '/principal/user:kyle',  
}
```

System principal can update identity

- Send update_identity to /principal/user:kyle from a built-in system principal.
- Assert identity/self is updated with trusted fields such as email, display_name, auth_provider, and auth_subject.

Non-system principal cannot update identity

- Send update_identity to /principal/user:kyle from /principal/user:alice.
- Assert the route rejects the request with 401/403.

Missing principal in production fails

- Simulate production/non-dev/non-insecure context.
- Spawn/send without Electric-Principal.
- Assert 401/400.

Dev fallback

- Simulate dev/insecure context.
- Spawn/send without header.
- Assert:

```
ctx.principal.url === '/principal/system:dev-local'  
created_by === '/principal/system:dev-local'
```

List by owner

- Spawn two agents under `user:kyle` and one under `user:alice`.
- Request:

GET `/_electric/entities?created_by=/principal/user:kyle`

- Assert only Kyle's entities are returned.

Implementation order

Phase 1 — Types and persistence

1. Add `principal.ts` parser/helpers.
2. Add `created_by` migration.
3. Update Drizzle schema.
4. Update entity types/public types.
5. Update registry create/read/list.

Phase 2 — Built-in principal type

6. Add `PostgresRegistry.ensureEntityType()`.
7. Add built-in principal identity state and `update_identity` inbox schemas.
8. Seed built-in principal entity type during server startup.

Phase 3 — Context and route behavior

9. Replace user-centric request context with principal-centric context:
 - `AuthenticatedRequestUser` → principal-oriented type
 - `ctx.authenticatedUser` → `ctx.principal`
 - `update/rename authenticated-user-format.ts` if still needed
10. Wire header extraction and dev fallback.
11. Require principal for all API routes.
12. Lazy-materialize `/principal/*` in `withExistingEntity()`.
13. Ensure inbound principal during spawn/send.
14. Persist `created_by` on spawn.
15. Use principal as `from` for send/initial messages/schedules.
16. Enforce the built-in `update_identity` send restriction for principal entities.

Phase 4 — Runtime handler context

17. Include `createdBy/principal` in webhook enrichment.
18. Expose `ctx.principal` in runtime handler context.

Phase 5 — Tests/docs

19. Add parser tests.
20. Add spawn/send/materialization/list tests.
21. Add identity initialization and `update_identity` authorization tests.
22. Update agents development docs with header/trust-boundary/dev fallback notes.

Implementation decisions

1. All API routes require a principal. Missing principal is an error, except local/dev mode where the server supplies `system:dev-local`.
2. Internal routes/code paths should include a principal too. There should be one principal-aware entry point into verbs.
3. Request/body from must not spoof principals. Use `ctx.principal.url` as the sender; reject mismatches.
4. `created_by` is immutable.
5. The `principal` entity type is immutable from user/API code and may only be created/modified by system code.
6. Principal identity lives in built-in `identity/self` state.
7. Principal identity updates use the built-in `update_identity` inbox message and are restricted to built-in system principals unless deployment policy explicitly allows otherwise.